



M.Tech Project



INTEROPERABLE SUPPLY-CHAIN MANAGEMENT SYSTEM USING BLOCKCHAIN & ZERO KNOWLEDGE PROOFS

Name: Sanket Phadtare

Branch: CI

Roll-No: CI24M12

Guide: Dr. Vandana Dhingra

Introduction

1. Current Supply Chain in the Market

Modern supply chain systems mostly depend on centralized databases and intermediaries for product tracking. While they record product movement from origin to consumer, they lack transparency, making it difficult to fully trust the stored data.

2. Misuse and Fraud in Supply Chains

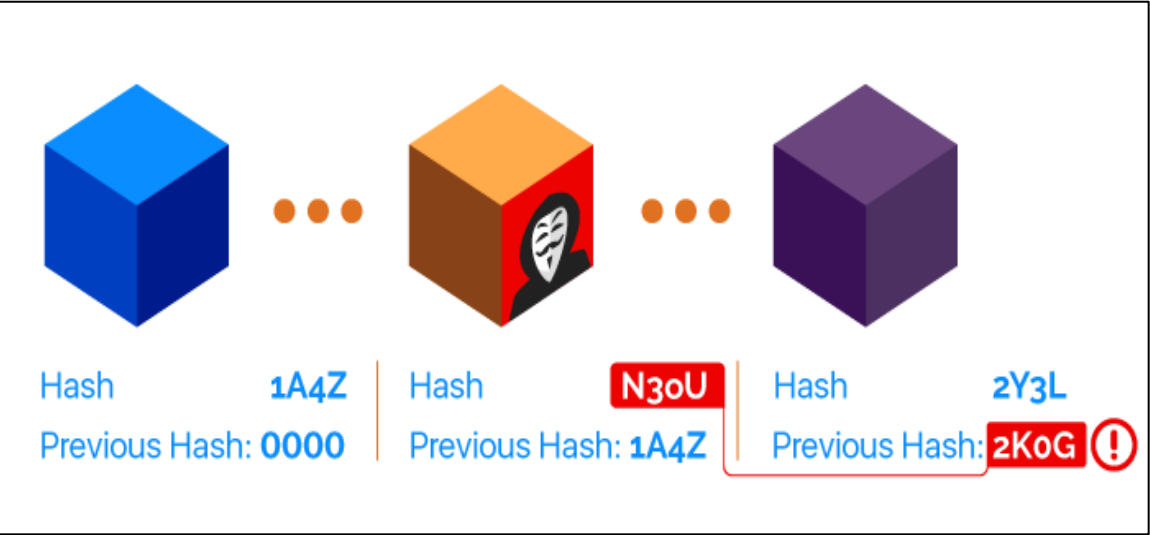
Centralized systems are vulnerable to data tampering, manipulation, and unauthorized changes. This leads to counterfeit products, false certifications, financial losses, health risks, and reduced consumer trust.

3. Available Solutions to Avoid Misuse

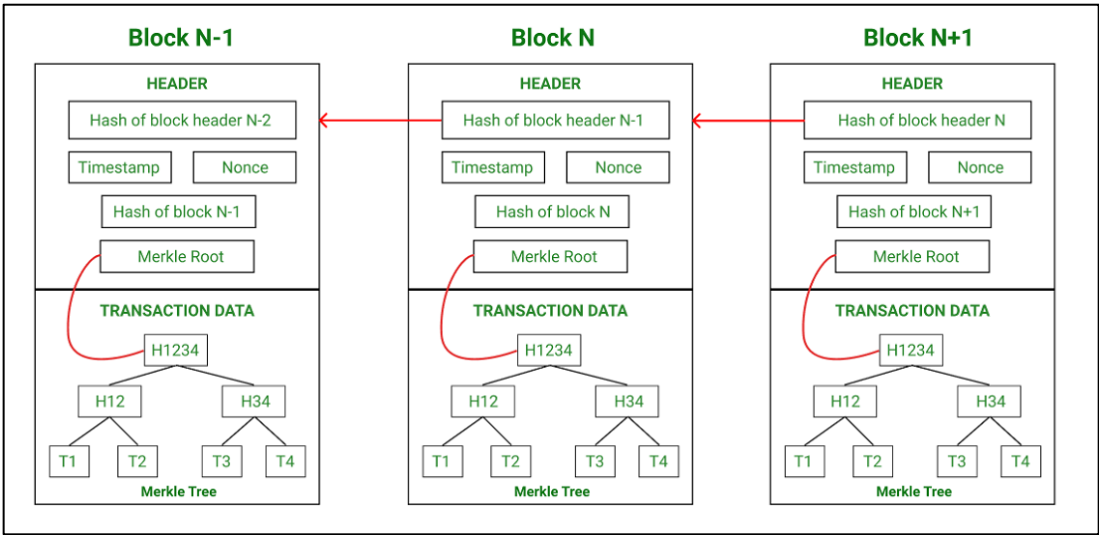
Blockchain technology provides a decentralized and immutable ledger for supply chain data. It ensures transparency, traceability, and tamper-proof storage of product records, reducing the chances of fraud.

4. Interoperable SCM

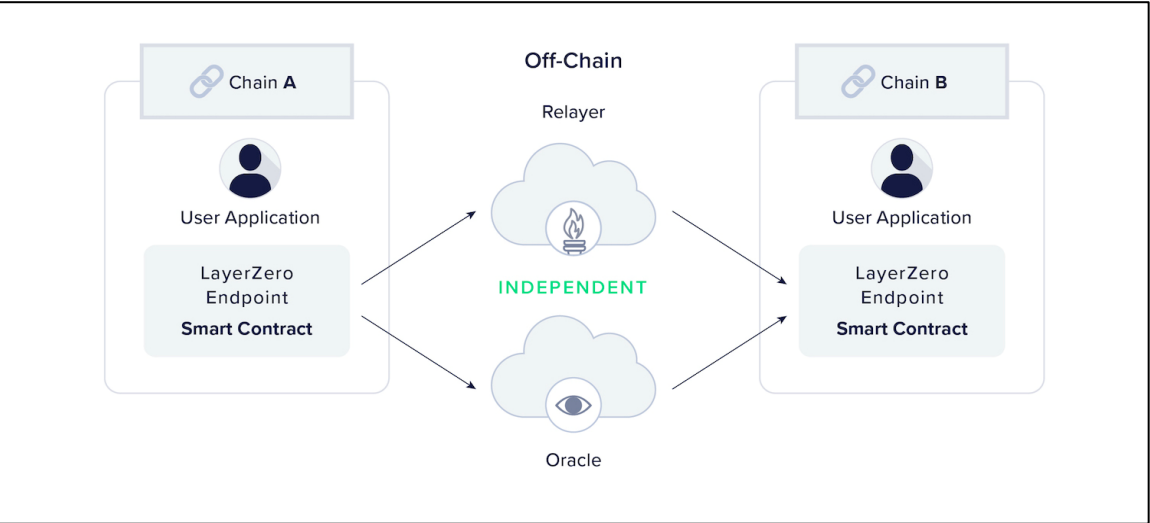
Our system combines blockchain with Zero Knowledge Proofs (ZKP) to overcome privacy and cost issues. It enables secure and private verification of product authenticity without exposing sensitive data, ensuring a trusted and efficient supply chain.



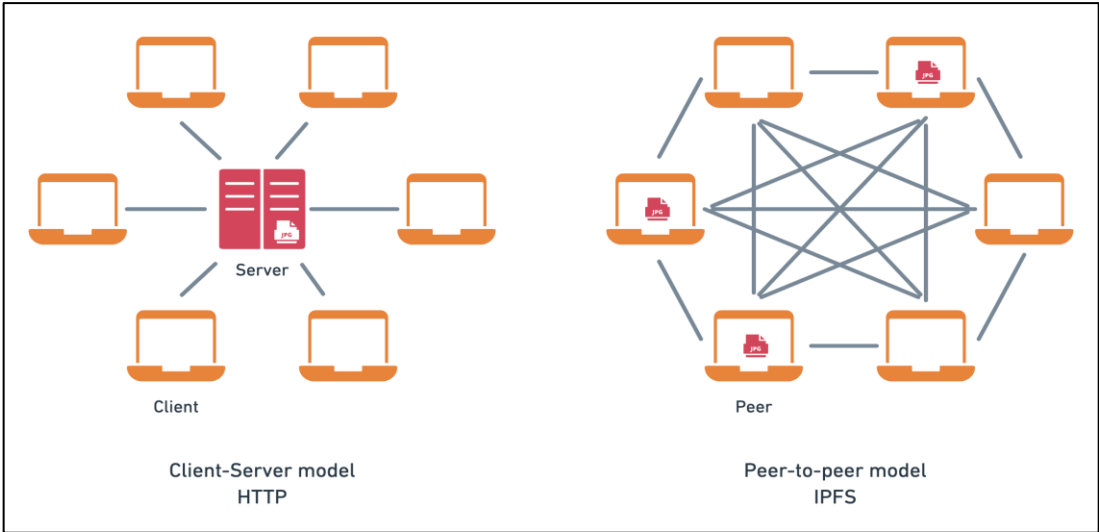
Blockchain Hash Linking



Merkle Root



Off-Chain Data Transfer



InterPlanetary File System (IPFS)

Zero Knowledge Proof



Alice

1) Alice claims "I am +18 years old!"

2) Bob says prove it!



3) Alice shows (sends) her valid ID card issued by an authority



Bob

4) Bob checks the originality and validity of the ID card and based on her age either **accepts** or **rejects**



Alice

1) Alice claims "I am +18 years old!"

2) Bob says prove it!

A masked ID card

3) Alice sends **a response generated with Alice's valid ID card** issued by an authority



Bob

4) Bob checks **the messages received from Alice** and returns either **accepts** or **rejects**.



Prover

1) Commitment

2) Challenge

3) Response



Verifier

Prover

$$r \leftarrow Z_q, a = g^r$$

$$z = r + c\omega$$

$$x = g^\omega$$

1) Commitment: $a = g^r$

2) Challenge: c

3) Response: z

Verifier



$$c \leftarrow \{0,1\}$$

$$g^z =? a \cdot x^c$$

Global Scandal: The Dangerous Trade in Fake Cancer Drugs

The global trade in fake cancer drugs has reached alarming proportions, with substandard and counterfeit medications infiltrating supply chains in more than 100 countries. Recent investigations, including a major report by the Bureau of Investigative Journalism, reveal that vital chemotherapy drugs—used to treat common cancers such as breast, ovarian, and leukemia—often fail quality tests, leaving patients at risk of ineffective treatment or dangerous side effects. In some cases, pills contain so little active ingredient that they are essentially useless; in others, excessive amounts pose severe health hazards. This crisis exposes critical weaknesses in global regulatory systems and supply chains, disproportionately affecting low- and middle-income countries where cancer incidence is rising rapidly.



The latest from Amazon's Counterfeit Crimes Unit: Amazon takes legal action against massive trademark fraud scheme

The Counterfeit Crimes Unit (CCU) is a global team dedicated to partnering with law enforcement, brands, and other stakeholders to disrupt counterfeiters and their networks. Learn more about the CCU, and continue to visit this page for the latest news and updates.



Problem Statement

- Existing blockchain-based supply chain and product authentication systems depend on off-chain servers to verify Merkle roots stored on the blockchain. This reduces decentralization and breaks the trustless nature of blockchain systems.
- Verification of product authenticity often requires access to sensitive raw data. As a result, stakeholders must rely on centralized auditors or APIs, which reintroduces trust issues and single points of failure.
- Additionally, current solutions lack interoperability across blockchains. There is no privacy-preserving, on-chain mechanism that allows different networks to verify data authenticity securely.
- Therefore, a Zero-Knowledge-Based framework is required to enable direct, trustless, and privacy-preserving verification without exposing sensitive supply chain data.

Objectives

1. To design a blockchain interoperability framework that allows seamless and trustless verification of Merkle roots between Polygon and Binance Chain.
2. To eliminate relayers by enabling direct on-chain verification of product authenticity through zero-knowledge proofs.
3. To ensure data privacy and security during cross-chain verification without revealing product details or salts used in Merkle root computation.
4. To enhance scalability and efficiency of cross-chain verification by implementing a ZK-based verification circuit.
5. To validate the proposed framework by demonstrating real-time product authenticity verification between Polygon and Binance blockchain.

Literature Review

Title	Techniques / Models	Performance Measures	Key Points	Limitations
Cosmos IBC Protocol Whitepaper arXiv Preprint (arXiv.org) 2020 [1]	IBC protocol Relayer-based cross-chain messaging Light client verification	Cross-chain latency Message reliability Fault tolerance	Secure blockchain-to-blockchain communication No centralized intermediary Supports cross-chain asset transfer	Complex implementation Limited ecosystem adoption Higher latency than single chain
Distributed Ledger Technologies in Supply Chain Security Management: A Comprehensive Survey. IEEE Transactions on Engineering Management 2021 [2]	Blockchain, DLT Smart contracts Cryptographic hashing	Traceability Data integrity Security	Ensures end-to-end supply chain traceability Prevents data tampering Improves trust among stakeholders	Scalability issues High cost & latency Privacy concerns

Literature Review

Title	Techniques / Models	Performance Measures	Key Points	Limitations
LayerZero: Trustless Omnichain Interoperability Protocol arXiv Preprint (arXiv.org) 2021 [3]	Omnichain architecture LayerZero protocol Oracle + Relayer model	Cross-chain message delay Gas efficiency Scalability	Direct cross-chain communication No wrapped tokens required Reduces interoperability overhead	Depends on oracle/relayer security Architecture complexity Not fully standardized
A survey on cross-chain technology: Challenges, development, and prospect. IEEE 2022 [4]	Blockchain (DLT) for supply chain Smart contracts Permissioned & public blockchains	Data immutability Transparency Traceability efficiency Security assurance (qualitative)	Blockchain improves trust and traceability in supply chains Reduces data tampering and fraud Useful in food, pharma, logistics sectors	Scalability issues High integration cost Lack of real-time performance evaluation
zkBridge: Trustless Cross-chain Bridges Made Practical 2022 [5]	Zero-Knowledge Proofs (ZKPs) zkSNARK-based verification Trustless cross-chain bridge	Verification cost Cross-chain latency Security against fraud	Enables trustless cross-chain communication No reliance on relayers or oracles Strong security using ZK proofs	High computational overhead Complex proof generation Higher implementation cost

Literature Review

Title	Techniques / Models	Performance Measures	Key Points	Limitations
Polyhedra Network: “zkBridge: Trustless Cross-Chain Verification 2023 [6]	Zero-Knowledge Proofs (zkSNARKs) Trustless cross-chain verification On-chain proof verification	Proof generation cost Cross-chain latency Security guarantee	Uses ZK proofs to verify messages across blockchains Eliminates trusted relayers/oracles High security — prevents forgery Enables trustless cross-chain asset/message transfer	High computational cost for proof generation Requires complex ZK setup Scalability challenges for frequent transfers

Research Gaps

There is no widely adopted standardized framework for generating and validating Zero-Knowledge Supply Chain Management Proofs (ZK-SCM Proofs) that function as a trust bridge between strict enterprise privacy requirements and public verification standards.

- 1) No existing solution enables cross-chain Merkle root verification between Polygon and other chains without intermediaries, while preserving privacy and decentralization.
- 2) Existing cross-chain verification methods rely on centralized bridges or relayers to exchange proof data, which introduces trust assumptions, latency, and potential points of failure.
- 3) Conventional verification mechanisms expose product or transaction data during validation, risking sensitive or proprietary information.

Methodology

1. Off-chain Data Hashing and Merkle Tree Construction:

Product attributes are individually hashed using salted Poseidon hashing to ensure privacy and ZKP compatibility. These hashes are structured into a fixed-depth Merkle Tree, where the root uniquely represents the entire dataset.

2. On-chain Storage and Proof Commitment:

The generated Merkle root and the IPFS CID (for product data) are stored in a Polygon smart contract as immutable records of authenticity.

3. Zero-Knowledge Proof Generation:

A ZK circuit is created (using tools like *Circom* and *snarkjs*) to prove the inclusion of a product's hash in the Merkle tree without revealing the actual data.

This proof can be verified on any blockchain.

4. Cross-Chain Verification (Polygon ↔ Binance):

The proof generated on Polygon can be submitted to another chain, where a ZK Verifier Contract validates the authenticity on-chain without off-chain help.

5. Result Validation and Display:

The verification result (Authentic / Tampered) is published on-chain and displayed to the user through a web interface.

Methodology

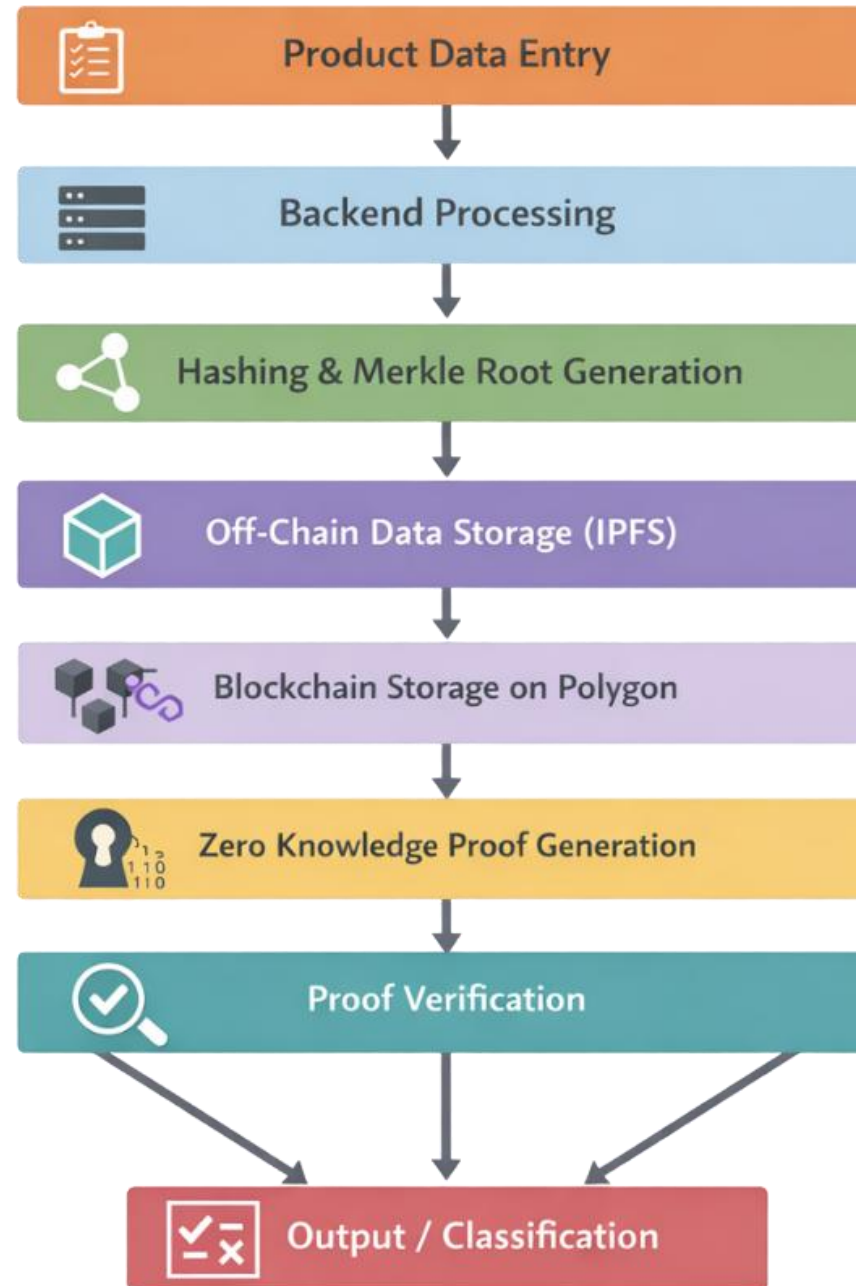


Fig.1

Architecture

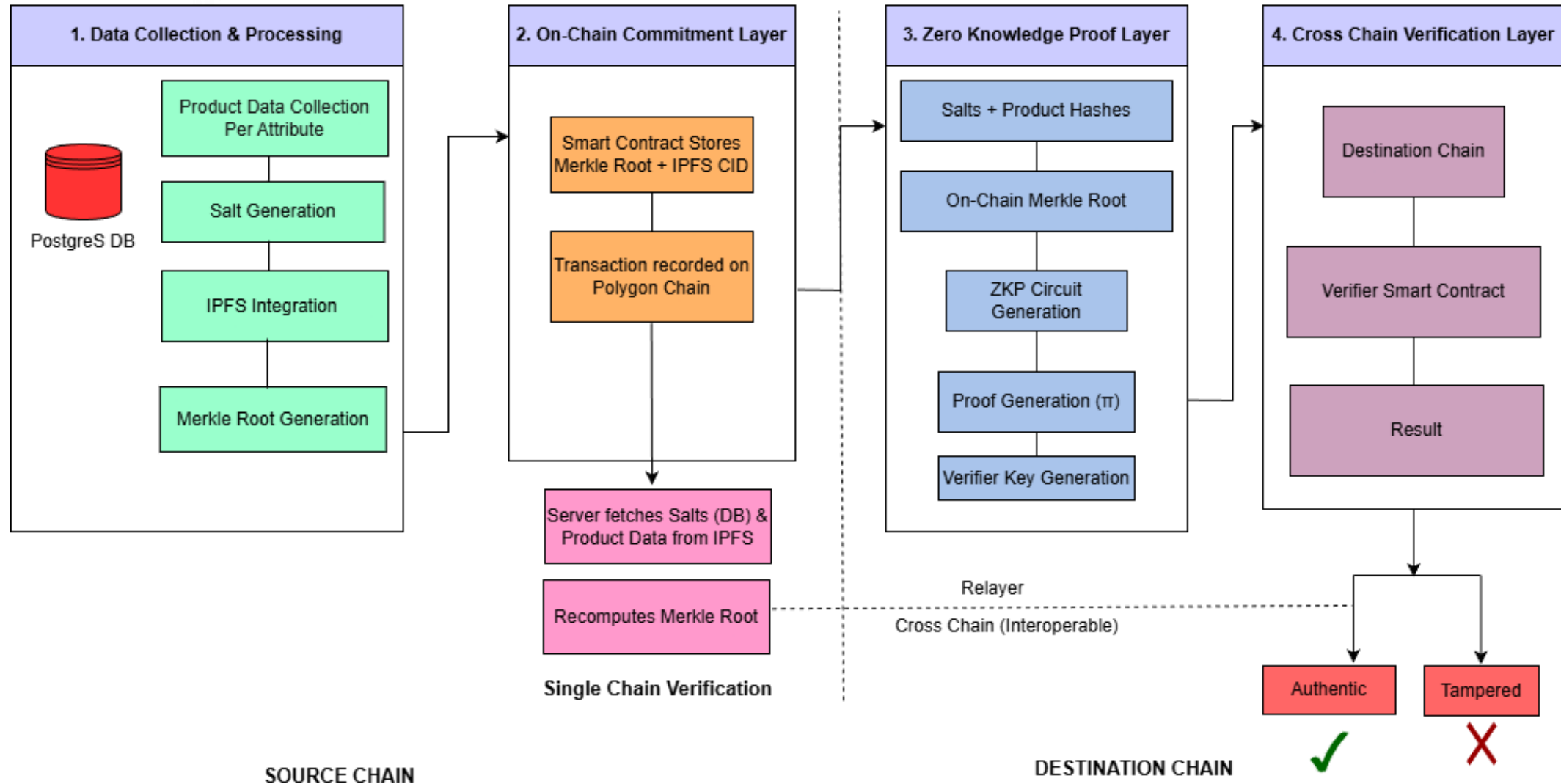
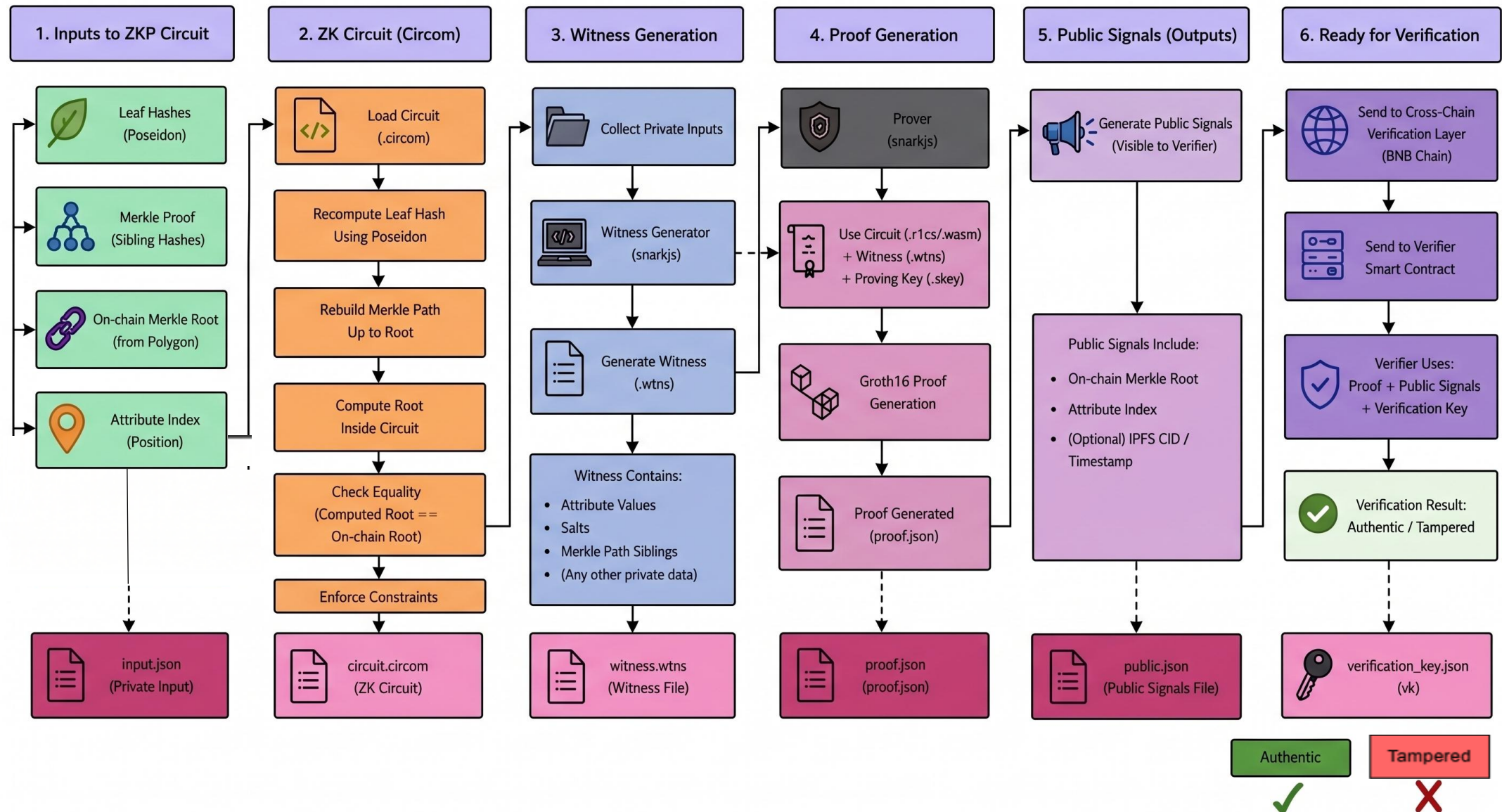


Fig.2

ZKP Architecture



EXPERIMENTS

Experiment	Input Size	Operation	Result	Observation
Data Storage (PostgreSQL + IPFS)	1 record	Store + Retrieve	Success	CID consistent, no data loss
Poseidon Hashing	10 attributes	Hash generation	Deterministic	Same input → same hash
Merkle Tree Construction	10 → padded to 16	Root generation	Success	Fixed-depth tree works correctly
Blockchain Storage (Polygon)	1 record	Store root + CID	Success	Low gas cost observed
Backend Verification	1 record	Recompute root	Match	Data integrity maintained
Tampering Detection	Modified data	Recompute root	Mismatch	System detects tampering

ZKP EXPERIMENTS

Experiment	Tool/Tech Used	Output	Result	Observation
Circuit Compilation	Circom	.r1cs, .wasm	Success	Circuit correctly built
Witness Generation	snarkjs	witness.wtns	Success	Failed when inputs incorrect
Proof Generation	Groth16	proof.json	Success	Fast proof generation
Local Verification	snarkjs	OK	Valid	Proof is mathematically correct
On-chain Verification	Verifier.sol (BNB)	true	Success	Required pB swap fix
Cross-chain Validation	Polygon → BNB	Verified	Success	Root matched across chains

CONTRIBUTION

1) Custom Circom Circuit for Merkle Root Validation

Designed and implemented a Circom circuit that:

- Takes private leaf hashes
- Reconstructs Merkle Tree
- Validates root against public input

2) Design of ZKP-Based Verification Architecture

Proposed and implemented a custom verification architecture combining:

- Merkle Tree commitments
- Off-chain storage
- Groth16-based proof validation

3) Proof-Based Cross-Chain Verification Model

- Generates ZKP proof from Polygon-anchored data
- Transfers proof across chains without data sharing
- Enables verification on BNB Chain using smart contract

OUTPUT

POS localhost:6000/api/pigs X

piglifecycle.sol

gear .env

☐ ...

POST ▾ http://localhost:6000/api/pigs

Send

Query

Headers²

Auth

Body¹

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

```
1  [
2    {
3      "pigId": 1,
4      "birthDate": "12-7-2025",
5      "soldAt": "20-12-2025",
6      "breed": "AtBirth",
7      "geneticLineage": "XYZ456",
8      "birthWeight": 1.8,
9      "earTag": "ET0001",
10     "sex": "F",
11     "status": "Alive",
12     "farmId": 102
13   }
14 ]
```

Status: 200 OK Size: 29 Bytes Time: 8.96 s

Response

Headers⁶

Cookies

Results

Docs

{}

≡

1 Batch data added successfully

```
{
  "level": "info",
  "message": "Server running on port 6000"
}
{"level": "info", "message": "Processing batch of pigs"}
{"level": "info", "message": "Data for pig 1 added to IPFS"}
{"level": "info", "message": "Inserting Data for pig 1 into DB"}
Total cost in INR: ₹NaN
{"level": "info", "message": "Batch transaction successful with hash: 0xd712efed0833927365fd216a01d28c33b4d30df8071008fa8c915ec37363582b"}
{"level": "info", "message": "Batch processed: 1 pigs added"}
```

```
postgres=# select * from pig_profiles;
 pig_id | birth_date | sold_at | breed | genetic_lineage | birth_weight | ear_tag | sex | status | farm_id |      created_at      |      updat
ed_at    |      salt1      |      salt2      |      salt3      |      salt4      |      salt5      |      salt6      |      salt7      |      salt8      |      salt9      |      salt10      |      salt11
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
      1 | 2025-07-12 | 20-12-2025 | AtBirth | XYZ456          |          1.8 | ET0001 | F   | Alive  |      102 | 2026-01-08 12:16:07.373212 | 2026-01-08 12
:16:07.373212 | 1d18c6f3f815ef17b421f8489462f826 | 8c3994d8039948554512d5489242d462 | 103374222130f9e73fbfa8a44874cd97 | 317b7ec746997f69a956823320409598 |
793245855a2c877b71aac8d44567e6fd | aa0f4265eb93ac305acaf74762da539f | a8be0febcd82deed4ad88a31df495237 | cd37bd8a64525de11c04332c34220752 | e252244f3ea83b17
110b51d885b12a09 | 1474074e8a5ccdfd8ae4a87949b65a1a
(1 row)
```

```
postgres=# select * from qr_codes;
 qr_id | pig_id | qr_code_data |      created_at
-----+-----+-----+-----
    135 |      1 | MQ==         | 2026-01-08 12:16:07.373212
(1 row)
```

 Pinata

Workspace

Sanket's Workspace

IPFS

Files

Groups

Gateways

Analytics

FILES

PUBLIC

PRIVATE

Public files are accessible via IPFS

🔍

Search files and CIDs

☐	NAME	CID	SIZE	CREATION DATE	FILE ID
☐	No name 	QmU1y...UMWJE 	188 B	1/8/2026	20  

SINGLE CHAIN PRODUCT VERIFICATION

POS

localhost:6000/api/verify.. X

piglifecycle.sol

.env

...

POST ▾

http://localhost:6000/api/verify/pig

Send

Query

Headers²

Auth

Body¹

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1 {

2 "p_qr_code": "MQ=="

3 }

Status: 200 OK

Size: 55 Bytes

Time: 5.88 s

Response

Headers⁶

Cookies

Results

Docs

{}

≡

1 {

2 "message": "Pig Data is Authentic",

3 "status": "Verified"

4 }

```
{"level": "info", "message": "Please wait while we verify Pig details..."}  
{"level": "info", "message": "Pig Details Verified (Hit Cache): Pig Data is Authentic", "status": "Verified"}
```

localhost:6000/api/verify.. X

piglifecycle.sol

.env

POST

http://localhost:6000/api/verify/pig

Send

Query

Headers 2

Auth

Body 1

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1 {

2 "p_qr_code": "MQ=="

3 }

Status: 200 OK

Size: 55 Bytes

Time: 3 ms

Response

Headers 6

Cookies

Results

Docs

1 {

2 "message": "Pig Data is Authentic",

3 "status": "Verified"

4 }

CROSS-CHAIN PRODUCT VERIFICATION

THUNDER CLIENT

New Request

Activity

filter activity

POST localhost:6000/api/pigs

just now

POST localhost:6000/api/verify/pig

2 months ago

POST localhost:6000/api/verify

2 months ago

POST localhost:3000/api/send

6 months ago

POST localhost:3000/api/mint-from-attest...

8 months ago

POST localhost:3000/api/readings/1/verify

8 months ago

POST localhost:3000/api/readings

8 months ago

POST localhost:3000/api/farmers

8 months ago

POST localhost:1000/api/webhook/tokens

8 months ago

POST localhost:1000/api/getbalance/:wall...

8 months ago

POST localhost:1000/api/registeruser

8 months ago

localhost:6000/api/pigs

POST http://localhost:6000/api/pigs

Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1  [  
2    {  
3      "pigId": 2,  
4      "birthDate": "12-7-2025",  
5      "soldAt": "20-12-2025",  
6      "breed": "AtBirth",  
7      "geneticLineage": "XYZ456",  
8      "birthWeight": 2,  
9      "earTag": "ET0002",  
10     "sex": "M",  
11     "status": "Alive",  
12     "farmId": 102  
13   }  
14 ]
```

Format

Status: 200 OK Size: 29 Bytes Time: 10.25 s

Response Headers 6 Cookies Results Docs

1 Batch data added successfully

Response Preview Chart

Leaf Hashes Used: [

```
'19465704262466163408715769316185127341360364119772489479101156845471344098096',  
'6966960697804169595113301214669155325832531313633474150976391631561475963111',  
'17987869983568040014896963796977631578217001604994024271574445657717764297184',  
'4678497767531798189391113932985095792489710815445706199242000412696763773633',  
'7257010178749575214494318731003814492454703029918185414039355378003330256134',  
'17990618297691446258851657561069292171659270690475010615107638629949574444497',  
'7277626029584021201980851414503420429288999203753418593843952813988589100737',  
'9995598682457261670092184211610709170216495861070134472934730507680921497271',  
'7319231774515184249998073120408169571585457157064935667081130585737724455888',  
'8017975512417968670292504783572310320769864128251015249772823873686919311123'
```

]

Merkle Root: 0x123aeb88124e2e3bb91a9f95c142184ce45262496f9569508b8903ffcd2cb95e

✓ input.json generated automatically

localhost:6000/api/pigs

proof.json

proof.json > ...

```
1 {  
2   "pi_a": [  
3     "9896357707147020413456461109945629683747294935197244318512739982517409587646",  
4     "5325110793259430877554419476131260741083570895417054286774312587309593919470",  
5     "1"  
6   ],  
7   "pi_b": [  
8     [  
9       "14132609348051588907423134830899653639701312632706402074832655501173893583058",  
10      "2873573446765829206434144544739889369409312370310183747993729102166528150626"  
11    ],  
12    [  
13      "18272260743258498112190123738504415225823898780446231991696527149337079410145",  
14      "16489080195677697131615948914761579222537786823517284455086267593241397615221"  
15    ],  
16    [  
17      "1",  
18      "0"  
19    ]  
20  ],  
21  "pi_c": [  
22    "13931231288896982516146744236146007109959378984541454830476325684225175741291",  
23    "9634353433916509631261999215916465146292888742664807927318450349710413536036",  
24    "1"  
25  ],  
26  "protocol": "groth16",  
27  "curve": "bn128"  
28 }
```

```
C:\Windows\System32\cmd.e X + v
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

D:\Sanket\A.T Pvt.Ltd\Tasks\Task-4\website> node -e "console.log(BigInt('0x123aeb88124e2e3bb91a9f95c142184ce45262496f9569508b8903ffcd2cb95e').toString()))"
8245733983127966679752293374606168278760272199238826624840445102798163196254

D:\Sanket\A.T Pvt.Ltd\Tasks\Task-4\website>node pig_js/generate_witness.js pig_js/pig.wasm input.json witness.wtns

D:\Sanket\A.T Pvt.Ltd\Tasks\Task-4\website>npx snarkjs groth16 prove pig_final.zkey witness.wtns proof.json public.json

D:\Sanket\A.T Pvt.Ltd\Tasks\Task-4\website>npx snarkjs zkey export verificationkey pig_final.zkey verification_key.json
[INFO] snarkJS: EXPORT VERIFICATION KEY STARTED
[INFO] snarkJS: > Detected protocol: groth16
[INFO] snarkJS: EXPORT VERIFICATION KEY FINISHED

D:\Sanket\A.T Pvt.Ltd\Tasks\Task-4\website>npx snarkjs groth16 verify verification_key.json public.json proof.json
[INFO] snarkJS: OK!
```

DEPLOYMENT OF VERIFIER CONTRACT BINANCE NETWORK

The image shows the Remix Ethereum IDE interface with the following components:

- Top Bar:** Browser address bar showing `remix.ethereum.org/#lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.26+commit.8a97fa7a.js`. Navigation icons and a search bar are present.
- Left Panel (DEPLOY & RUN TRANSACTIONS):**
 - Environment:** Shows "WalletConnect" and "Account 1" with address `0x5B3...eddC4` and a balance of 0.000 ETH.
 - Deploy:** Set to "Main (1) network". The contract "Groth16Verifier" (Verifier.sol) is listed as "Compiled".
 - Value:** Set to 0.
 - Gas limit:** Set to "auto" (0).
 - Deploy Button:** A large blue button labeled "Deploy".
 - Deployed Contracts:** Shows 0 contracts. A button "+ Add Contract" is available.
 - Transactions recorder:** Shows 0 transactions. Buttons for "Save" and a trash icon are present.
- Center Panel (Code Editor):** Displays the Solidity code for `Verifier.sol`. The code includes functions for storing data, checking pairing, and computing linear combinations.

```
72 mstore(mIn, x)
73 mstore(add(mIn, 32), y)
74 mstore(add(mIn, 64), s)
75
76 success := staticcall(sub(gas(), 2000), 7, mIn, 96,
77
78 if iszero(success) {
79     mstore(0, 0)
80     return(0, 0x20)
81 }
82
83 mstore(add(mIn, 64), mload(pR))
84 mstore(add(mIn, 96), mload(add(pR, 32)))
85
86 success := staticcall(sub(gas(), 2000), 6, mIn, 128,
87
88 if iszero(success) {
89     mstore(0, 0)
90     return(0, 0x20)
91 }
92 }
93
94 function checkPairing(pA, pB, pC, pubSignals, pMem) ->
95     let _pPairing := add(pMem, pPairing)
96     let _pVk := add(pMem, pVk)
97
98     mstore(_pVk, IC0x)
99     mstore(add(_pVk, 32), IC0y)
100
101     // Compute the linear combination vk_x
102
103     g1_mulAccC(_pVk, IC1x, IC1y, calldataload(add(pubS
104
105
106     // -A
107     mstore(_pPairing, calldataload(pA))
108     mstore(add(_pPairing, 32), mod(sub(q, calldataload(add(pA, 32))), q))
109
110     // B
```

Right Panel (Wallet Overview):

- Balance: **\$0.00** (+\$0.00 (+0.00%)) with a "Discover" link.
- Buttons: Buy, Swap, Send, Receive.
- Trade section: "Trade tokenized Big Tech" with a sub-header "Tokens".
- Selected Token:** BNB Smart Chain Testnet. The token "tBNB" is selected, showing a balance of 0.0146 tBNB.
- Footer: "remix.ethereum.org" and "A.T" logo.

Bottom Bar: Includes an "AI copilot" toggle and an "Explain contract" button.

FINAL VERIFICATION ON BINANCE NETWORK

REMIX 2.1.0

DEPLOY & RUN TRANSACTIONS

Environment Reset

Browser Extension <select>

Account 16 0xdaf...bbbc1 0.013 tBNB

0xDd5...44978 12min ago

Functions

Select a function to interact with...

Low level interaction

verifyProof uint256[2], uint256[2][2], uint256[2], uint256[1]

[98963577071470204134564611099456296837472949351972443:

[[28735734467658292064341445447398893694093123703101837

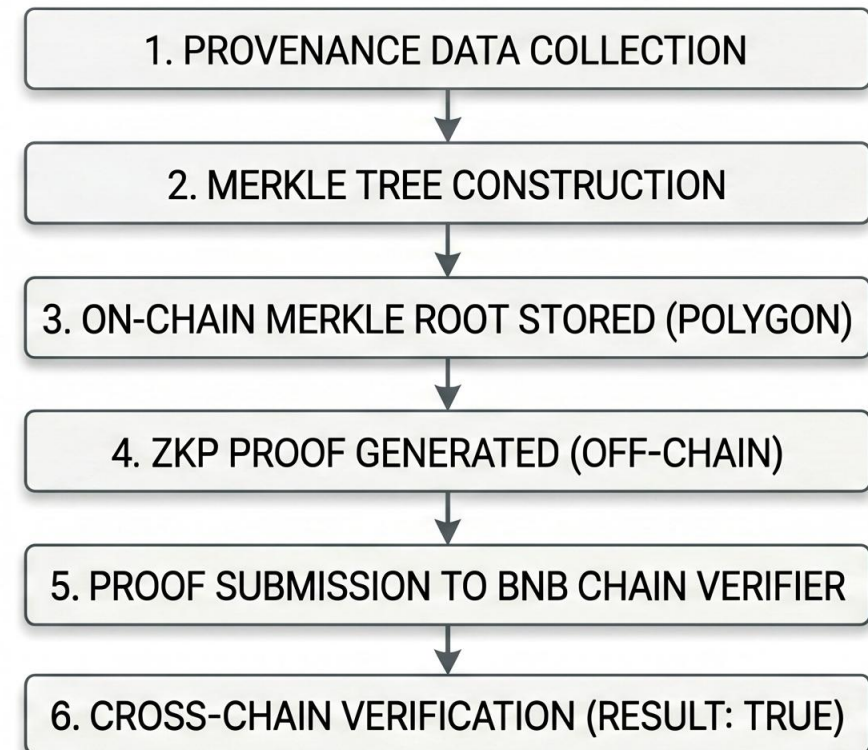
[13931231288896982516146744236146007109959378984541454:

[82457339831279666797522933746061682787602721992388266

o:bool: true

Call

Verification Status: TRUE
Product Status: Authentic
Chain Used: BNB Smart Chain



Future Work

- Enabling Streamline Automation ZK circuit implementation for Real-Time Corss-Chain Verification.
- Integrate Multi-Chain verifier contracts.
- Extend framework to support other EVM-compatible chains.
- Generalization of Framework: Adapting the ZK-SCM Proof primitive for deployment in diverse high-regulation, high-value industries (e.g., Pharmaceuticals, Aerospace component tracking, Luxury Goods provenance).
- Plug-and-Play Connectors: Developing reusable, standardized integration modules (R3 Corda, other enterprise ERP systems) to accelerate the adoption of this privacy-preserving verification mechanism.

Conclusion

- Achieved secure and tamper-proof supply chain verification using blockchain
- Ensured data privacy through salted hashing and Zero Knowledge Proofs
- Reduced on-chain cost by storing only Merkle Root + IPFS CID
- Successfully implemented Poseidon-based Merkle Tree for ZKP compatibility
- Enabled end-to-end data integrity verification without exposing raw data
- Achieved cross-chain interoperability (Polygon → BNB Chain verification)
- Automated proof generation and verification pipeline

Project Timeline

PHASES	WORK
Phase-1 (August-September)	Literature Review, Centralized SCM Development
Phase-2 (October-November)	Smart Contract Anchoring, ZKP Setup, IPFS Client Setup & Verifier Smart Contract Development
Phase-3 (December-February)	Decentralized SCM development, ZKP Circuit Designing, Cross Chain Data Transfer
Phase-4 (March-April)	Evaluation

References

- [1] Kwon, Jae, and Ethan Buchman. "Cosmos whitepaper." *A Netw. Distrib. Ledgers* 27 (2019): 1-32.
- [2] Asante, Mary, Gregory Epiphaniou, Carsten Maple, Haider Al-Khateeb, Mirko Bottarelli, and Kayhan Zrar Ghafoor. "Distributed ledger technologies in supply chain security management: A comprehensive survey." *IEEE Transactions on Engineering Management* 70, no. 2 (2021): 713-739.
- [3] Zarick, Ryan, Bryan Pellegrino, and Caleb Banister. "Layerzero: Trustless omnichain interoperability protocol." *arXiv preprint arXiv:2110.13871* (2021).
- [4] Mao, Hanyu, Tiezheng Nie, Hao Sun, Derong Shen, and Ge Yu. "A survey on cross-chain technology: Challenges, development, and prospect." *Ieee Access* 11 (2022): 45527-45546.
- [5] Xie, Tiancheng, Jiaheng Zhang, Zerui Cheng, Fan Zhang, Yupeng Zhang, Yongzheng Jia, Dan Boneh, and Dawn Song. "zkbridge: Trustless cross-chain bridges made practical." In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pp. 3003-3017. 2022.
- [6] Polyhedra Network: "zkBridge: Trustless Cross-Chain Verification," 2023.